

Clustering Part 2

Homework

Prof. Bisbee

Due Date: 2024-11-14

Tweets, Texts, and Topics, oh my!

Lots of press attention (and academic/commercial research) to social media communication. Trying to characterize (in the hopes of predicting) behavior and “sentiment.” Many are trying to do this at scale – analyzing the behavior of large numbers of individual users – including using clustering methods to uncover different “types” of users that can be used to understand and predict human opinion and behavior across a wide range of activities and concepts.

Perhaps no individual has been more scrutinized than former President Trump – who was arguably defined by his use of twitter as a medium of communication, agenda-setting, and mobilization. Multiple news stories and academic papers have focused on the corpus of Trump Tweets.

For example, the NY Times did an expose here (<https://www.nytimes.com/interactive/2019/11/02/us/politics/trump-twitter-presidency.html>) whereby they had reporters read every single tweet and classify it. (See the methodology here (<https://www.nytimes.com/2019/11/02/insider/trump-twitter-data.html>)). Remarkably, this was work that was done by hand. Hopefully by the end of class you can see how you could do something similar using R! More similar to what we are going to do is the work by fivethirtyeight (<https://fivethirtyeight.com/features/the-worlds-favorite-donald-trump-tweets/>).

We used to be able to read in Trump data via the Twitter API, but since that has been deactivated we are going to use data that was collected and posted here (<https://www.kaggle.com/austinreese/trump-tweets>).

Note that you could also look at news coverage using data that people have collected on the scrolling chyrons at the bottom of the screen here (<https://archive.org/details/third-eye>).

As an example of a recent publication in *Nature: Humanities and Social Sciences Communications* using sentiment analysis see: Sentiments and emotions evoked by news headlines of coronavirus disease (COVID-19) outbreak (<https://www.nature.com/articles/s41599-020-0523-3>).

Let's load the packages we are going to use into memory. You may need to install some of these/

```
library(readr)
library(tidyverse)
library(lubridate)
library(scales)
```

Just to give you a sense of the preprocessing, here I read in a csv file and did some manipulations

```

trumptweets <- read_csv("../data/trumptweets.csv")
#View(trumptweets)
glimpse(trumptweets)
tweets <- trumptweets %>%
  select(id, content, date, retweets, favorites) %>%
  mutate(date = with_tz(date, "EST"),
         Tweeting.hour = format(date, format = "%H"),
         Tweeting.date = format(date, format = "%Y-%m-%d"),
         Tweeting.date = as.Date(Tweeting.date),
         Tweeting.year = as.factor(format(date, format = "%Y")))

```

First thing we always do is to see what we have so we know what we are working with and what we have to do to answer the questions we want to answer. Next we select the variables we want and practice creating some time objects for future use. (We are using the `lubridate` package for the date manipulations.) Note that `date` had the time of the tweet in UTC so we used the `with_tz` function from `lubridate` package to change the time zone to Eastern Standard Time (as that is where Washington, DC, Bedminster, NJ, and Mar Lago, FL are located) – note that dates are also changed if the timezone change crosses days (a benefit of saving the object as a date object!).

Because our data is larger than usual, we want to keep an eye on how much is loaded into memory. Since we no longer need `trumptweets` let us remove it via `rm()`.

```
rm(trumptweets)
```

Let's focus on asking the question of how Trump's Twitter behavior changed over time. This is a broad question, so let's break it up into a few specific questions to tackle using the tools we have talked about thus far to help demonstrate that based on the concepts and code we have talked about in class you are able to do quite a bit.

1. How did the frequency of Trump's tweets change over time – and, in particular, once he was elected President.
2. When did Trump tweet? Was "Executive Time" a change to his behavior or did he always have a self-defined "Executive Time"?
3. Which of his tweets were most impactful? (Measures via Retweets and Favorites).

All of this can be done at the level of the tweet using tools we have previously used and discussed in class – i.e., no text analysis required. So we will start there. Sometimes simple analyses will get you what you need and complexity for the sake of complexity is not a virtue.

After using the data – and conditional means – to answer these descriptive questions we can then pivot to analyze what was being tweeted about using several tools that will get into the analysis of the content of the text being tweeted.

4. What were the topics that Trump was tweeting about before and after becoming president? (`kmeans`)
5. What were the dominant "sentiments" being expressed by Trump and how did that change after becoming president. (Sentiment Analysis)

Note that we are going to compare pre-post presidency but you have the tools, and also the code based on what we do today, to do much finer-grained analyses. You can compare how the behavior changes each year. Or each month? Or in response to his impeachments. Or how his activity in the 2016 campaign compares to his activity in his 2020 campaign. Or dive into the 2016 campaign and see how he acted and reacted through the Republican

nomination during his rise to the Republican nomination. There are many, many fascinating (and largely unanswered) questions that you should be empowered to be able to do based on what we cover! We will dive deeper a few times, but largely to illustrate the amazing stuff that you can do.

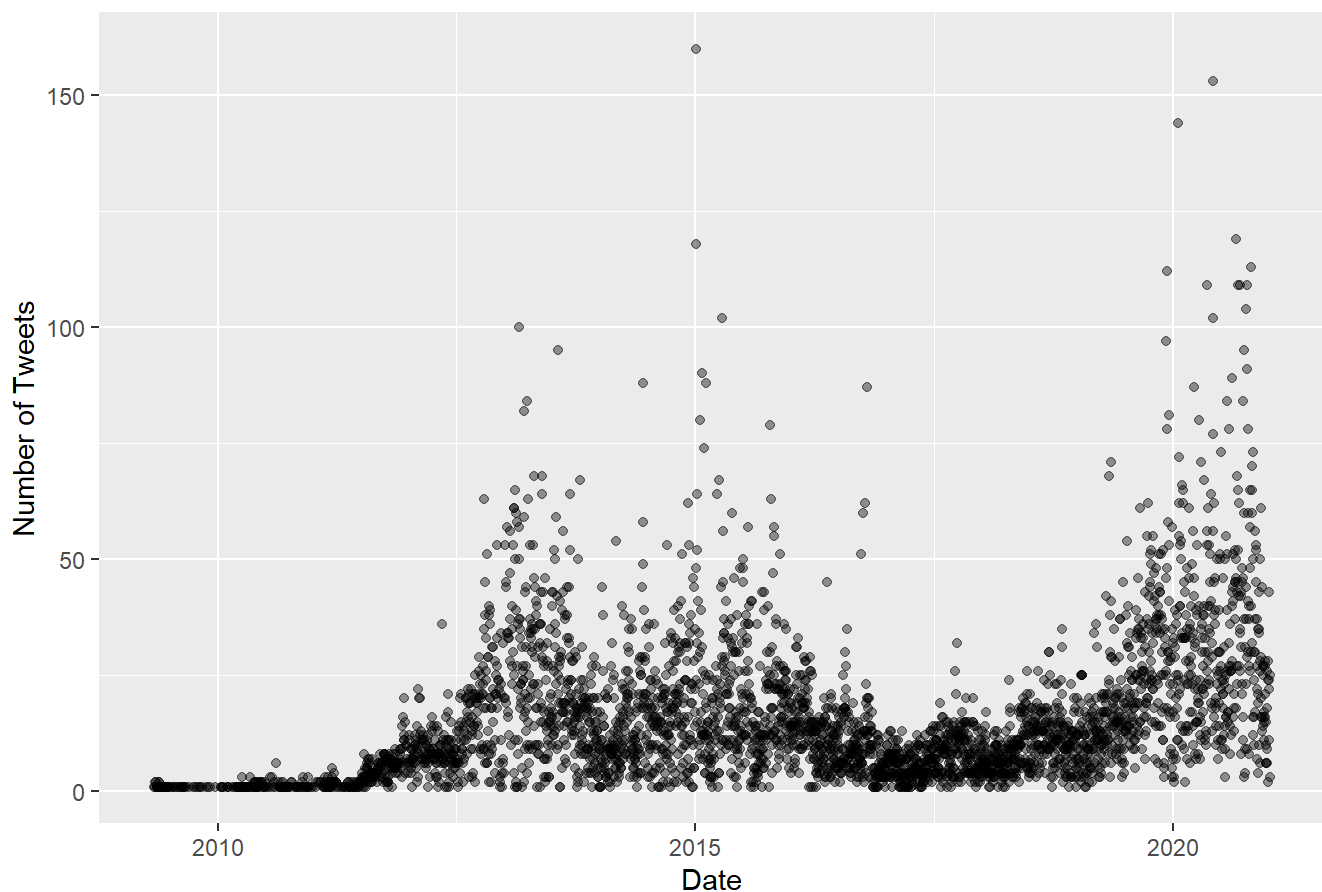
So let's get to it!

```
tweets <- read_rds(file="https://github.com/jbisbee1/DS1000_F2024/raw/main/data/Trumptweets.Rds")
```

So let's start by describing our data graphically. How frequently was President Trump tweeting throughout the time series we possess?

```
tweets %>%  
  group_by(Tweeting.date) %>%  
  count() %>%  
  ggplot() +  
  geom_point(aes(x=Tweeting.date,y=n),alpha=.4) +  
  labs(x="Date",y="Number of Tweets",title="Tweeting by Trump")
```

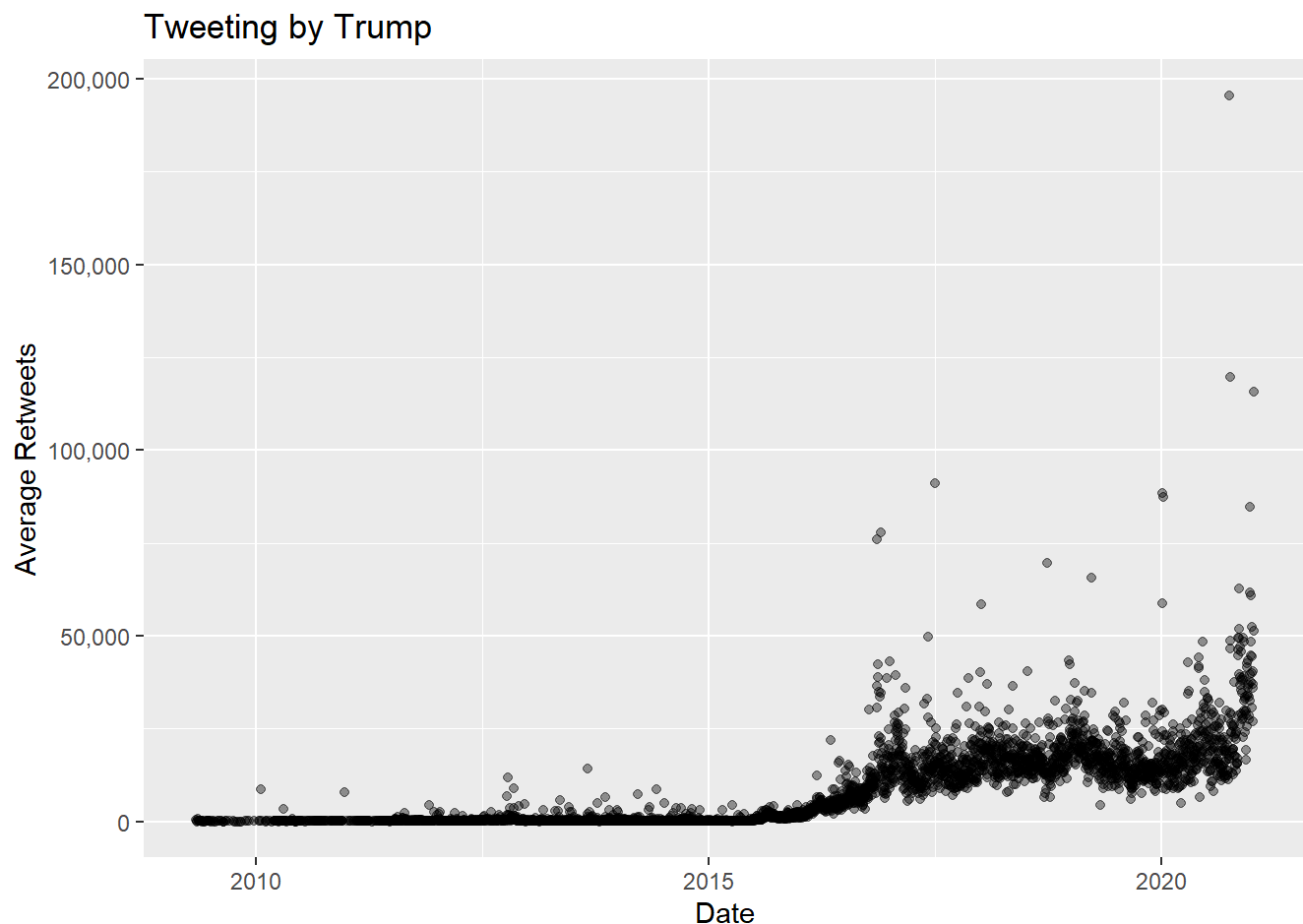
Tweeting by Trump



Here each point is the number of tweets in a day. Some days there was clearly a lot of activity. Moreover we can see that Trump was more active before becoming President and his frequency of tweeting increased over his presidency.

We can also consider how did the number of retweets, on average, change across days on which a tweet occurred? (Here we are using the `scales` library to set the `scale_y_continuous` to label numbers with commas (`label=comma`).)

```
tweets %>%
  group_by(Tweeting.date) %>%
  summarize(AvgRetweet = mean(retweets)) %>%
  ggplot() +
  geom_point(aes(x=Tweeting.date,y=AvgRetweet),alpha=.4) +
  labs(x="Date",y="Average Retweets",title="Tweeting by Trump") +
  scale_y_continuous(label=comma)
```



Clearly there is a lot of variation here. Which tweets were re-tweeted the most?

```
tweets %>%
  select(content,retweets) %>%
  top_n(retweets,n=10) %>%
  arrange(-retweets)
```



```
## # A tibble: 10 × 2
##   content                                retweets
##   <chr>                                <dbl>
## 1 "Tonight, @FLOTUS and I tested positive for COVID-19. We will begin... 408866
## 2 "#FraudNewsCNN #FNN https://t.co/WYUnHjjUjg" 293109
## 3 "TODAY WE MAKE AMERICA GREAT AGAIN!" 281289
## 4 "Are you allowed to impeach a president for gross incompetence?" 237674
## 5 "RT @SpaceX: Liftoff! https://t.co/DRBfdUM7JA" 235250
## 6 "A$AP Rocky released from prison and on his way home to the United ... 226235
## 7 "\"Why would Kim Jong-un insult me by calling me \"old,\"\" when ... 217199
## 8 "Be prepared, there is a small chance that our horrendous leadershi... 211345
## 9 "The United States of America will be designating ANTIFA as a Terro... 210615
## 10 "RT @realDonaldTrump: The United States of America will be designat... 210614
```

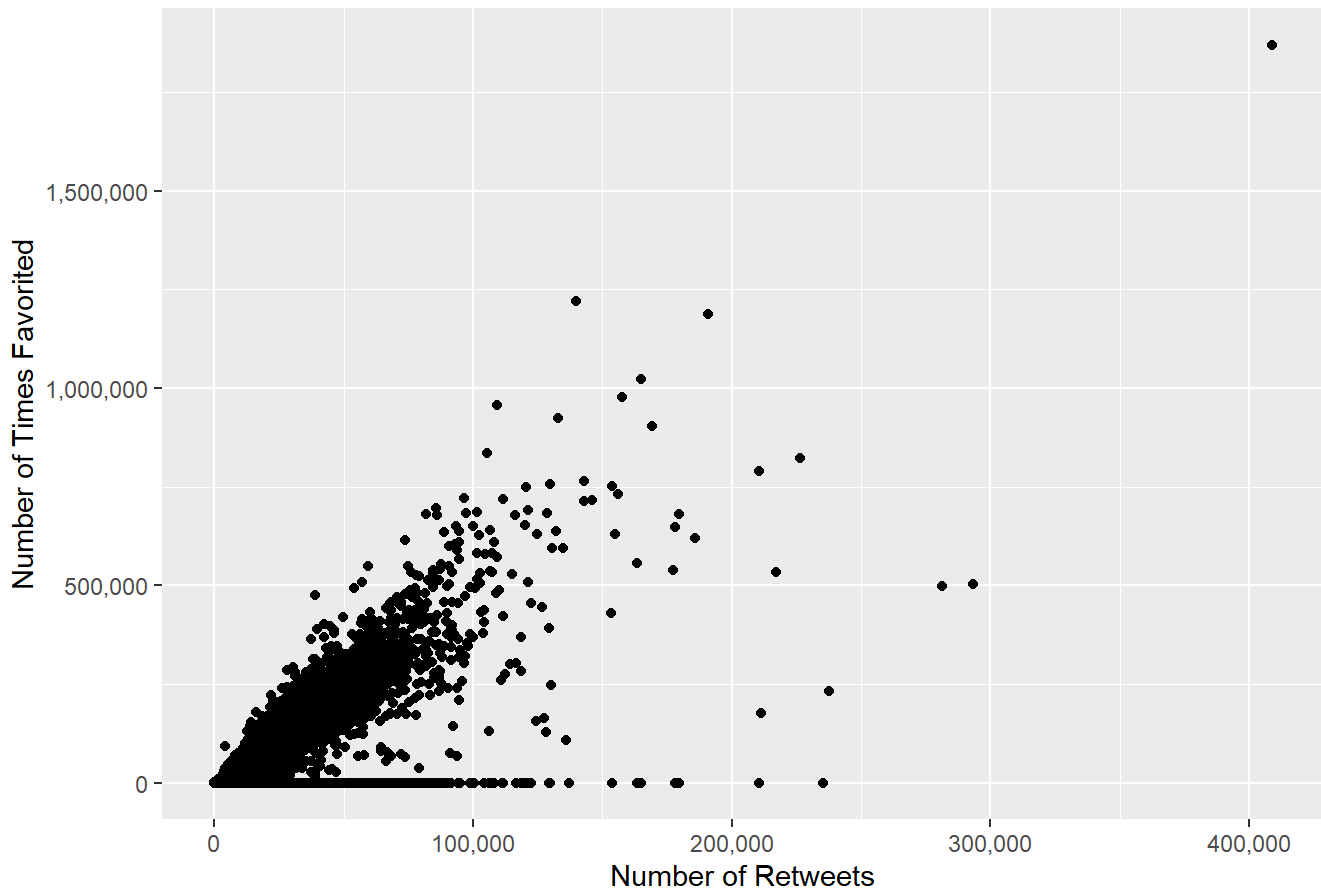
Now can you do the same to identify the tweets that were selected as a favorite? How does this list compare to the tweets that were most likely to be retweeted? Can you write this code?

```
# INSERT CODE HERE
```

In general, how should we measure influence/impact? Favorites or retweets? Does the choice matter? Let's look at the relationship to see how consequential the determination is.

```
tweets %>%
  ggplot(aes(x=retweets, y=favorites)) +
  geom_point() +
  scale_x_continuous(label=comma) +
  scale_y_continuous(label=comma) +
  labs(x= "Number of Retweets", y = "Number of Times Favorited", title="Trump Tweets: Relationship between Retweets and Favorites")
```

Trump Tweets: Relationship between Retweets and Favorites



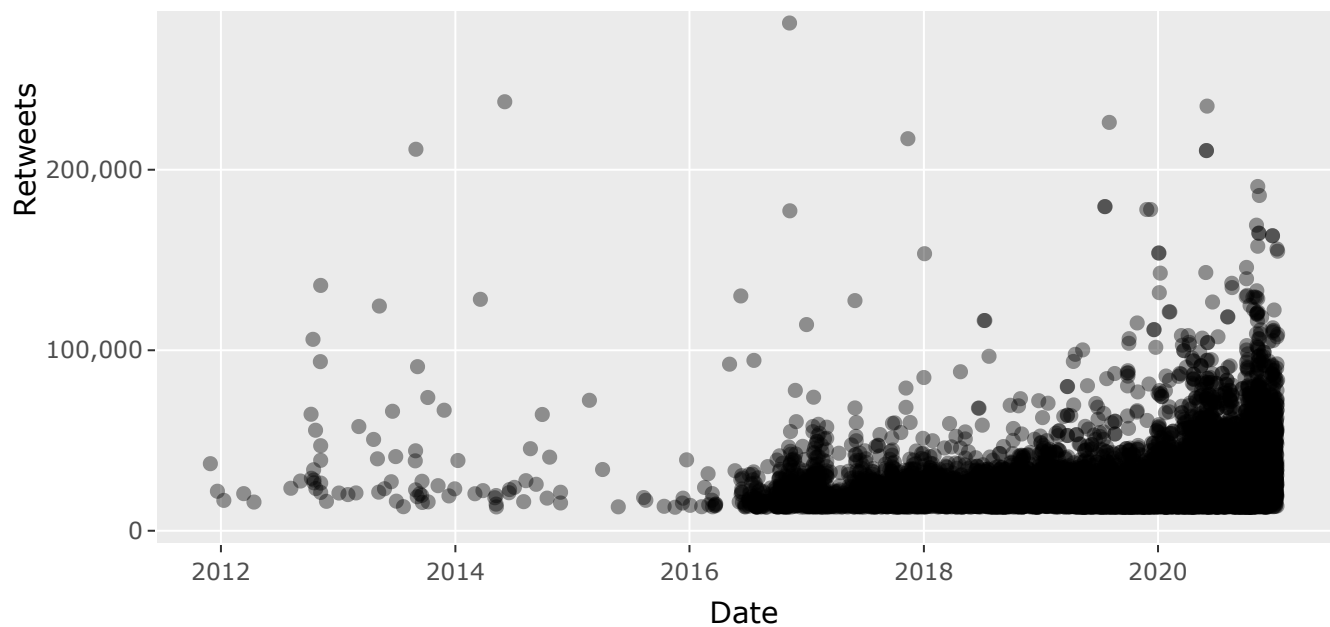
In general they seem pretty related, but there are a handful of tweets that are retweeted far more than they are favorited. (On your own, can you figure out which ones these are?)

We can also use `plotly` to create an HTML object with each point denoting how many retweets each tweet received and the date of the tweet. We can use this to label the tweets to get a better sense of what tweets were most re-tweeted? (This will be a very large plot given the number of tweets and the length of the content being pasted into the object! To keep things manageable let's focus on the top 75th-percentile of tweets.)

```
library(plotly)
gg <- tweets %>%
  filter(retweets > quantile(retweets, .75)) %>%
  ggplot(aes(x=Tweeting.date, y=retweets, text=paste(content))) +
  geom_point(alpha=.4) +
  labs(x="Date", y="Retweets", title="Tweeting by Trump") +
  scale_y_continuous(label=comma)
ggplotly(gg, tooltip = "text")
```

Tweeting by Trump



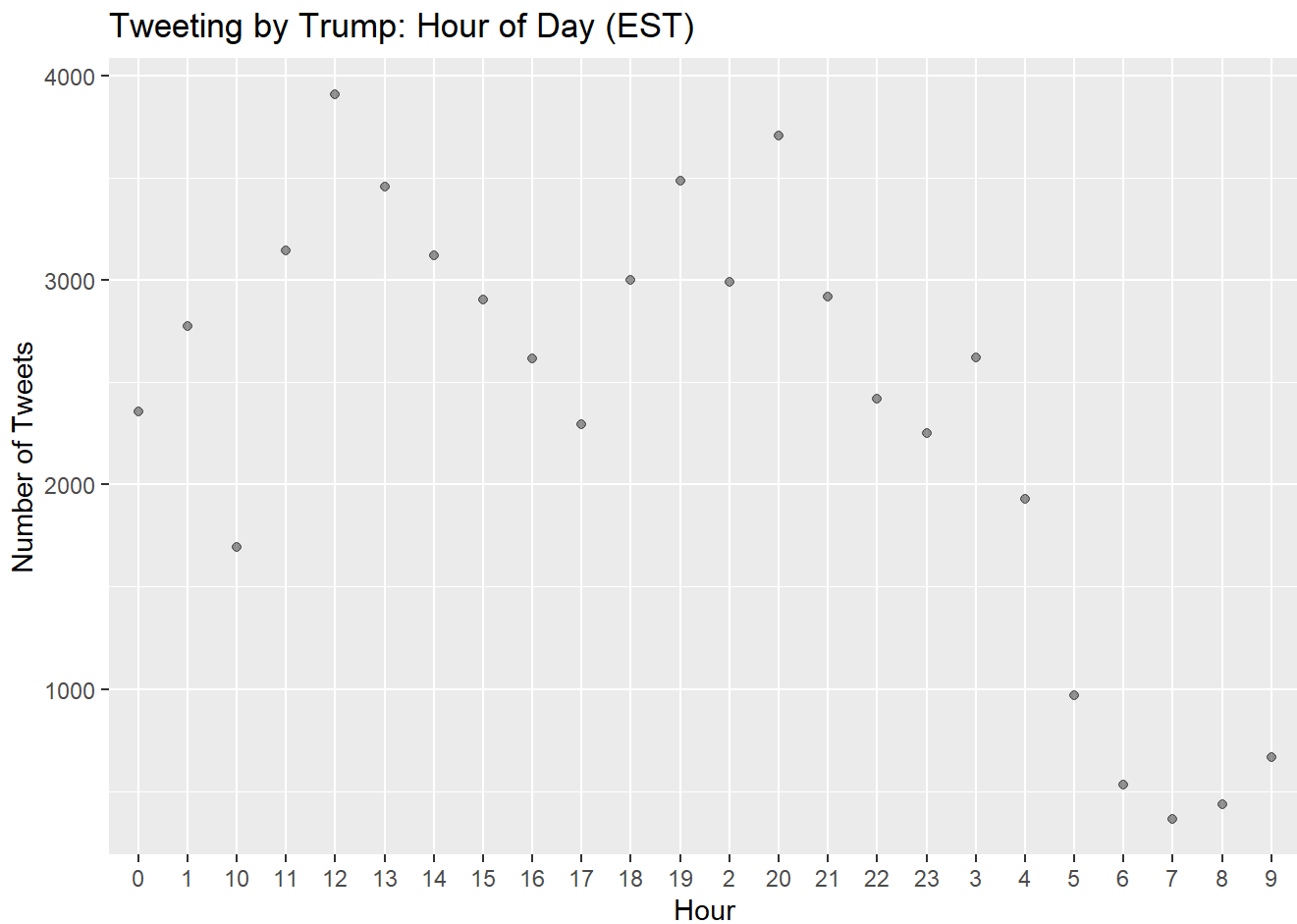


On your own, can you do the same for the most favorited tweets?

```
# INSERT CODE HERE
```

In addition to looking at the change over time we can also look at the hour at which Trump was tweeting using the `hour` variable we created. To start let's do the total number of tweets at each hour across the entire corpus.

```
tweets %>%
  group_by(Tweeting.hour) %>%
  count() %>%
  ggplot() +
  geom_point(aes(x=Tweeting.hour,y=n),alpha=.4) +
  labs(x="Hour",y="Number of Tweets",title="Tweeting by Trump: Hour of Day (EST)")
```

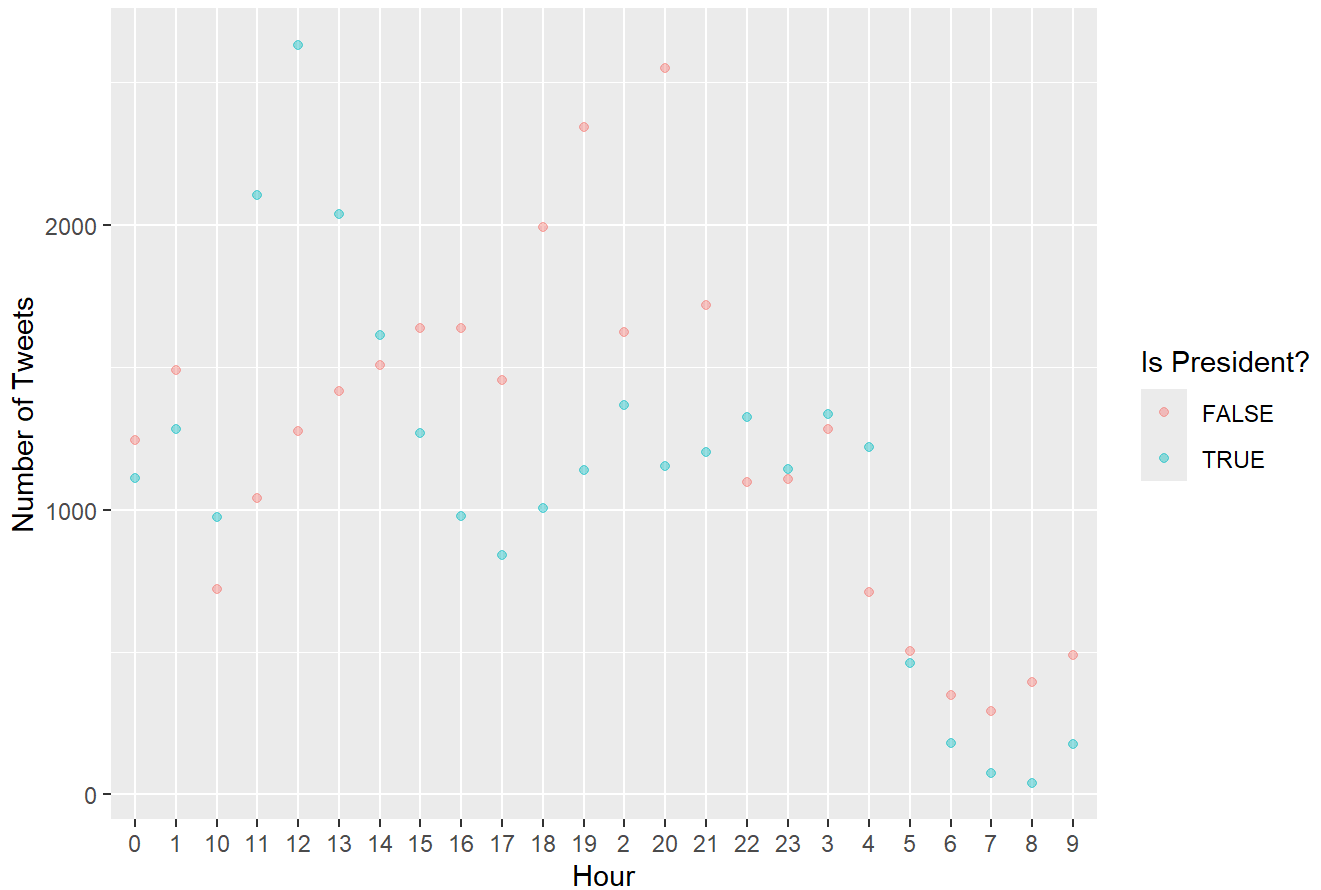


Did Trump's use to twitter change after he was elected President? Certainly we would think that the content might change – as well as how likely it was to be favorited and retweeted – but how about the frequency and timing of the tweets?

Let's create an indicator variable `PostPresident` using the `date` variable to define whether the date is before (`FALSE`) or after (`TRUE`) his election as president (we could also use the inauguration date, but some claimed that his behavior would change once he was officially projected.)

```
tweets %>%
  mutate(PostPresident = Tweeting.date > as.Date("2016-11-03")) %>%
  group_by(PostPresident, Tweeting.hour) %>%
  count() %>%
  ggplot() +
  geom_point(aes(x=Tweeting.hour, y=n, color=PostPresident), alpha=.4) +
  labs(x="Hour", y="Number of Tweets", title="Tweeting by Trump: Hour of Day (EST)", color
="Is President?")
```

Tweeting by Trump: Hour of Day (EST)

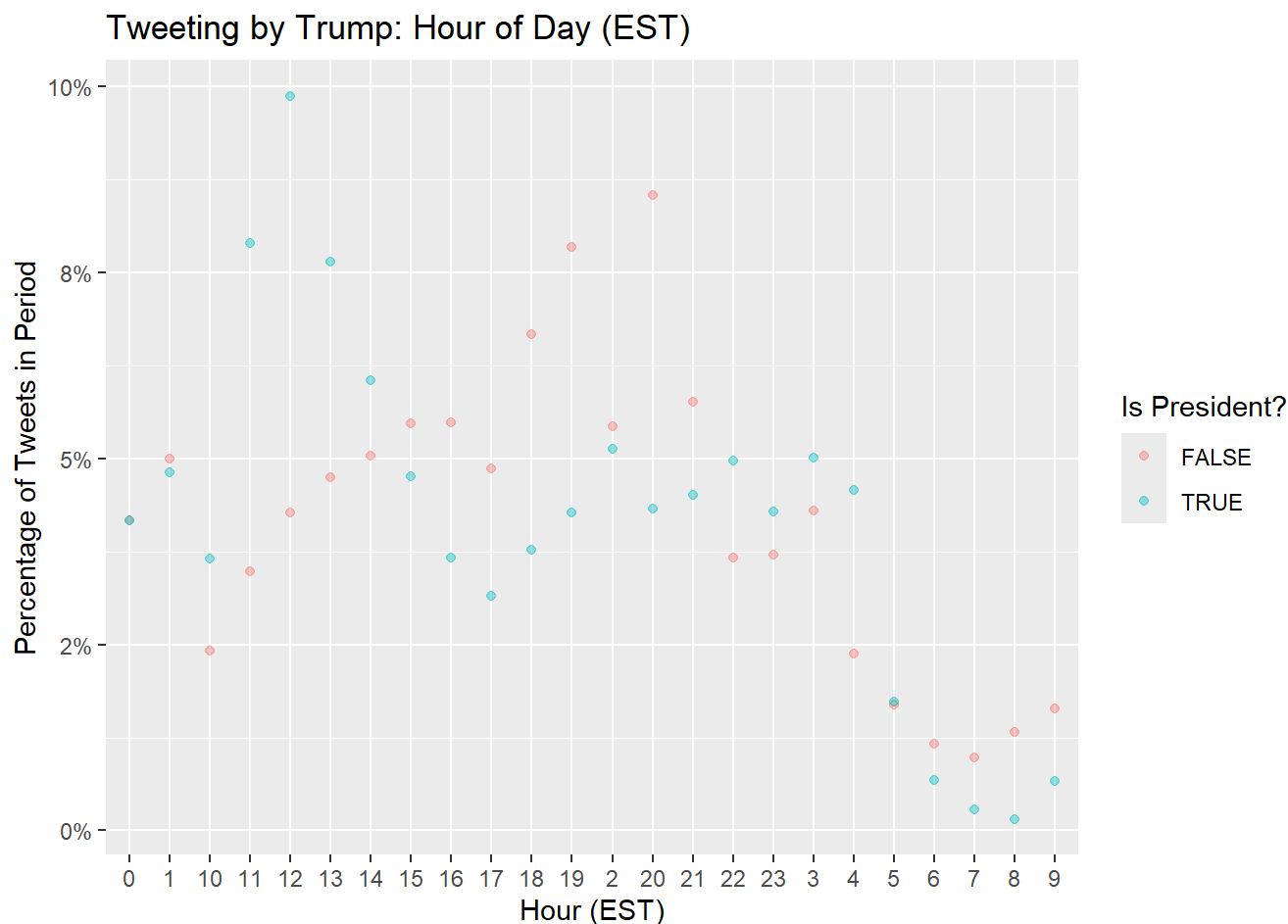


What do you observe?

But is it right to use the overall frequency? Do we prefer the proportion of tweets that were set at each hour pre/post presidency?

Let's use `mutate` to compute the proportion of tweets that occur at each hour in each period and then plot those using `color` to denote the pre/post time period.

```
tweets %>%
  mutate(PostPresident = Tweeting.date > as.Date("2016-11-03")) %>%
  group_by(PostPresident, Tweeting.hour) %>%
  count() %>%
  ungroup(Tweeting.hour) %>%
  mutate(Prop = n/sum(n)) %>%
  ggplot() +
  geom_point(aes(x=Tweeting.hour, y=Prop, color=PostPresident), alpha=.4) +
  labs(x="Hour (EST)", y="Percentage of Tweets in Period", title="Tweeting by Trump: Hour
of Day (EST)", color="Is President?") +
  scale_y_continuous(labels = scales::percent_format(accuracy = 1))
```

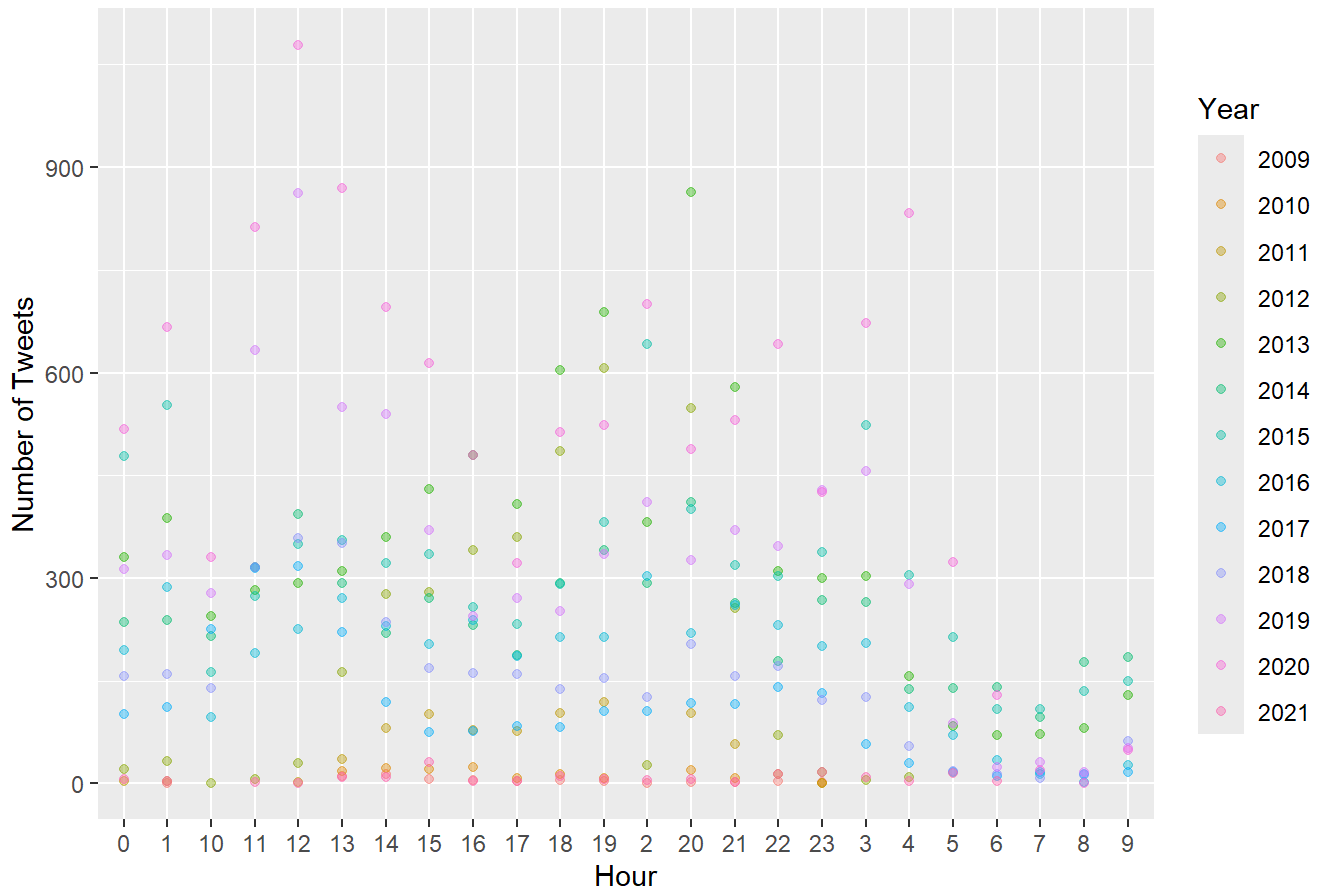


Hmm. A very different characterization! Always think about what the right calculation is. R will do what you tell it – usually ;) – but you need to think about what you are asking it to do!

We could also go deeper and look at variation by year. Here is a graph of the overall frequency. (Can you do the same for proportions?)

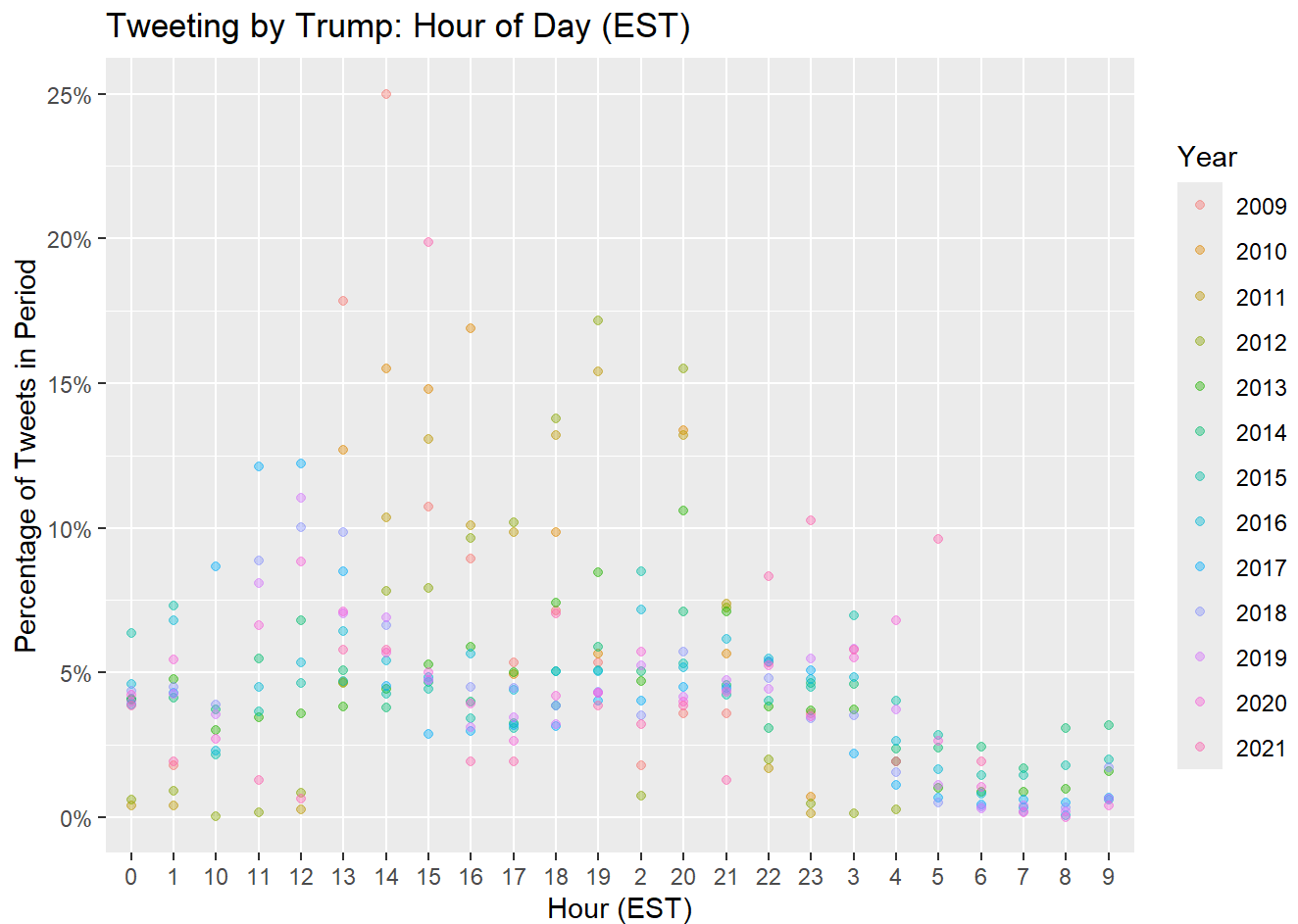
```
tweets %>%
  group_by(Tweeting.year, Tweeting.hour) %>%
  count() %>%
  ggplot() +
  geom_point(aes(x=Tweeting.hour, y=n, color=Tweeting.year), alpha=.4) +
  labs(x="Hour", y="Number of Tweets", title="Tweeting by Trump: Hour of Day (EST)", color
="Year")
```

Tweeting by Trump: Hour of Day (EST)



Can you graph the proportion of tweets per hour per year? How does that change your characterization. In your opinion, which is the more appropriate measure? The number of tweets or the proportion of tweets? Why or why not?

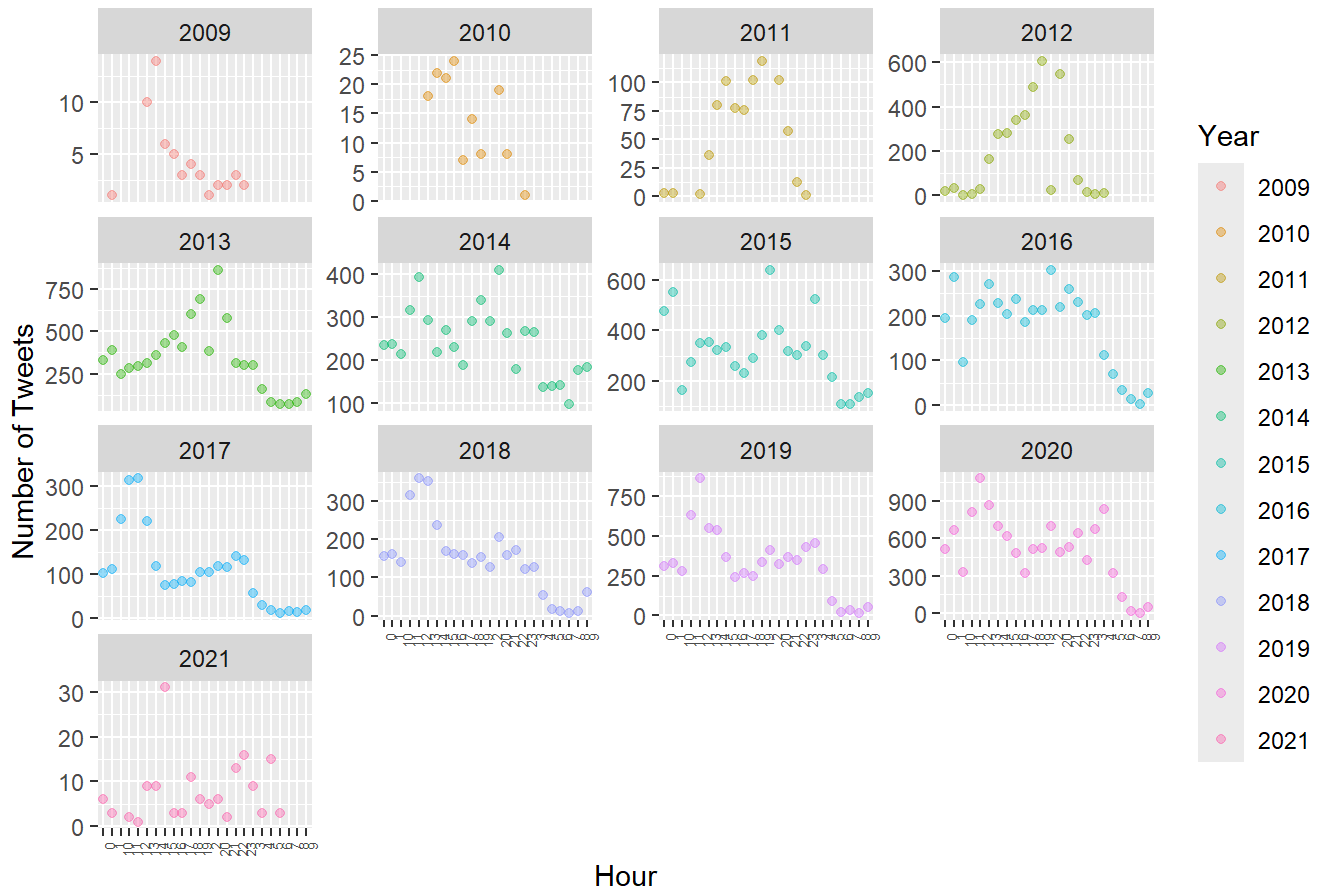
```
tweets %>%
  group_by(Tweeting.year, Tweeting.hour) %>%
  count() %>%
  ungroup(Tweeting.hour) %>%
  mutate(Prop = n/sum(n)) %>%
  ggplot() +
  geom_point(aes(x=Tweeting.hour, y=Prop, color=Tweeting.year), alpha=.4) +
  labs(x="Hour (EST)", y="Percentage of Tweets in Period", title="Tweeting by Trump: Hour
of Day (EST)", color="Year") +
  scale_y_continuous(labels = scales::percent_format(accuracy = 1))
```



Here we put everything on the same graph, but maybe it makes sense to create separate graphs - one for each value that we are using to define the `color`. To do this we just need to use a `facet_wrap` to create a bunch of graphs that will “wrap around” the screen starting from the lowest value of the facet defined after the `~` and arranged in a grid with `nrow=4` (Try different values here!). We have defined `scales = "free_y"` to let the graphs vary in the scales of the y-axis (because the frequencies vary). We could also choose `"fixed"` to give every graph the same x and y limits, or `"free_x"` to use the same y-axis scale and allow the x-axis to vary. `scale="free"` allows both the x and y axes to vary. Experiment with what happens if you change the scale. Why did I do what we did?

```
tweets %>%
  group_by(Tweeting.year, Tweeting.hour) %>%
  count() %>%
  ggplot() +
  facet_wrap(~ Tweeting.year, scales = "free_y", nrow = 4) +
  geom_point(aes(x=Tweeting.hour, y=n, color=Tweeting.year), alpha=.4) +
  theme(axis.text.x = element_text(angle = 90, hjust = 1, size=5)) +
  labs(x="Hour", y="Number of Tweets", title="Tweeting by Trump: Hour of Day (UTC)", color
="Year")
```


Tweeting by Trump: Hour of Day (UTC)



First try to do the same thing with the tweeting behavior Pre/Post Presidency. Can you use the `facet_wrap` to create that visualization? Which visualization do you prefer?

```
# INSERT CODE HERE
```

Now see if you can pull this together by plotting the time of tweeting by year but graphing the proportions this time. How should we define the `scales` in this case?

```
# INSERT CODE HERE
```

[OPTIONAL] Data Wrangling Text

You can skip this section and just load in the data in the next section, but this is if you want to know what I did to get the data from above ready for the analyses we do below. It involves working with the string variable `content` and deleting elements of that string to leave us only word “tokens.”

OK so that is what we can do at the level of the tweet. To analyze the content of the tweet we need to transform the string of each tweet into “tokens” (words) that we can then analyze. The first part is often the hardest – data-wrangling is typically 85% of the issue in data science. Rather than give you the cleaned data, here is a sense of what you need to do to make it work. Do not worry about understanding the content at this point.

The following is included for the interested student to get a sense of what is required to convert the `content` into tokens. Recall that our “data” looks like this:

```
tweets$content[1]
```

```
## [1] "Be sure to tune in and watch Donald Trump on Late Night with David Letterman as  
he presents the Top Ten List tonight!"
```

And we need to transform that into something we can analyse! This takes some work...

```
library(qdapRegex)  
library(tm)  
library(tidytext)  
library(SnowballC)
```

First we are going to strip out all of the url and twitter-formatted url from the tweet text using some pre-defined functions.

```
tweets <- tweets %>%  
  mutate(content = rm_twitter_url(content),  
         content = rm_url(content),  
         document = id)
```

Now we are going to write a function that takes as an argument a string (`x`) and then uses multiple functions to remove strings satisfying certain conditions.

First we are going to process the string `content` to remove combinations of letters/numbers that are not words. To do so we are going to define a function called `clean_tweets` and then apply it to the `content` variable in `tweets` tibble.

```

clean_tweets <- function(x) {
  x %>%
    # Remove mentions e.g. "@my_account"
    str_remove_all("@[[:alnum:]]_{4,}") %>%
    # Remove mentions e.g. "@ my_account"
    str_remove_all("@ [[:alnum:]]_{4,}") %>%
    # Remove hashtags
    str_remove_all("#[[:alnum:]]+") %>%
    # Remove hashtags
    str_remove_all("# [[:alnum:]]+") %>%
    # Remove twitter references
    str_remove_all("twitter[[:alnum:]]+") %>%
    # Remove twitter pics references
    str_remove_all("pictwitter[[:alnum:]]+") %>%
    # Replace "&" character reference with "and"
    str_replace_all("&", "and") %>%
    # Remove punctuation, using a standard character class
    str_remove_all("[[:punct:]]") %>%
    # Remove "RT: " from beginning of retweets
    str_remove_all("^RT: ") %>%
    # Replace any newline characters with a space
    str_replace_all("\\\\n", " ") %>%
    # Make everything lowercase
    str_to_lower() %>%
    # Remove any trailing whitespace around the text
    str_trim("both")
}

# Now apply this function to the `content` of `tweets`
tweets$content <- clean_tweets(tweets$content)

```

Now that we have pre-processed the `content` string lets do some more more wrangling to extract word “tokens” from this string and then also remove the tokens that are not meaningful words. Let’s also define the object `reg` containing all the various characters that an be used.

```

reg <- "([A-Za-z\\d#@']|'?![A-Za-z\\d#@'])"
tweet_words <- tweets %>%
  filter(!str_detect(content, '^\"')) %>%
  unnest_tokens(word, content, token = "regex", pattern = reg) %>%
  filter(!word %in% stop_words$word, str_detect(word, "[a-z]")) %>%
  mutate(word = str_replace_all(word, "\\d+", "")) %>% # drop numbers
  mutate(word = str_replace_all(word, "twitter[A-Za-z\\d]+|&", "")) %>%
  mutate(word = str_replace_all(word, "pictwitter[A-Za-z\\d]+|&", "")) %>%
  mutate(word = str_replace_all(word, "pic[A-Za-z\\d]+|&", "")) %>%
  mutate(word = str_replace_all(word, "pic", "")) %>%
  mutate(word = str_replace_all(word, "againpic[A-Za-z\\d]+|&", "")) %>%
  # mutate(word = wordStem(word)) %>%
  mutate(document = id) %>%
  select(-id) %>%
  filter(word != "") # drop any empty strings

```

Now use the `anti_join` to remove all `stop_words` to focus on the words with “content.”

```
data("stop_words", package = "tidytext")
tweet_words <- anti_join(tweet_words, stop_words, by = "word")
```

Back to Reality

We can also just read in the already-wrangled data `tweet_words` and proceed from here.

```
tweet_words <- read_rds(file="https://github.com/jbisbee1/DS1000_F2024/raw/main/data//Trump_tweet_words.Rds")
```

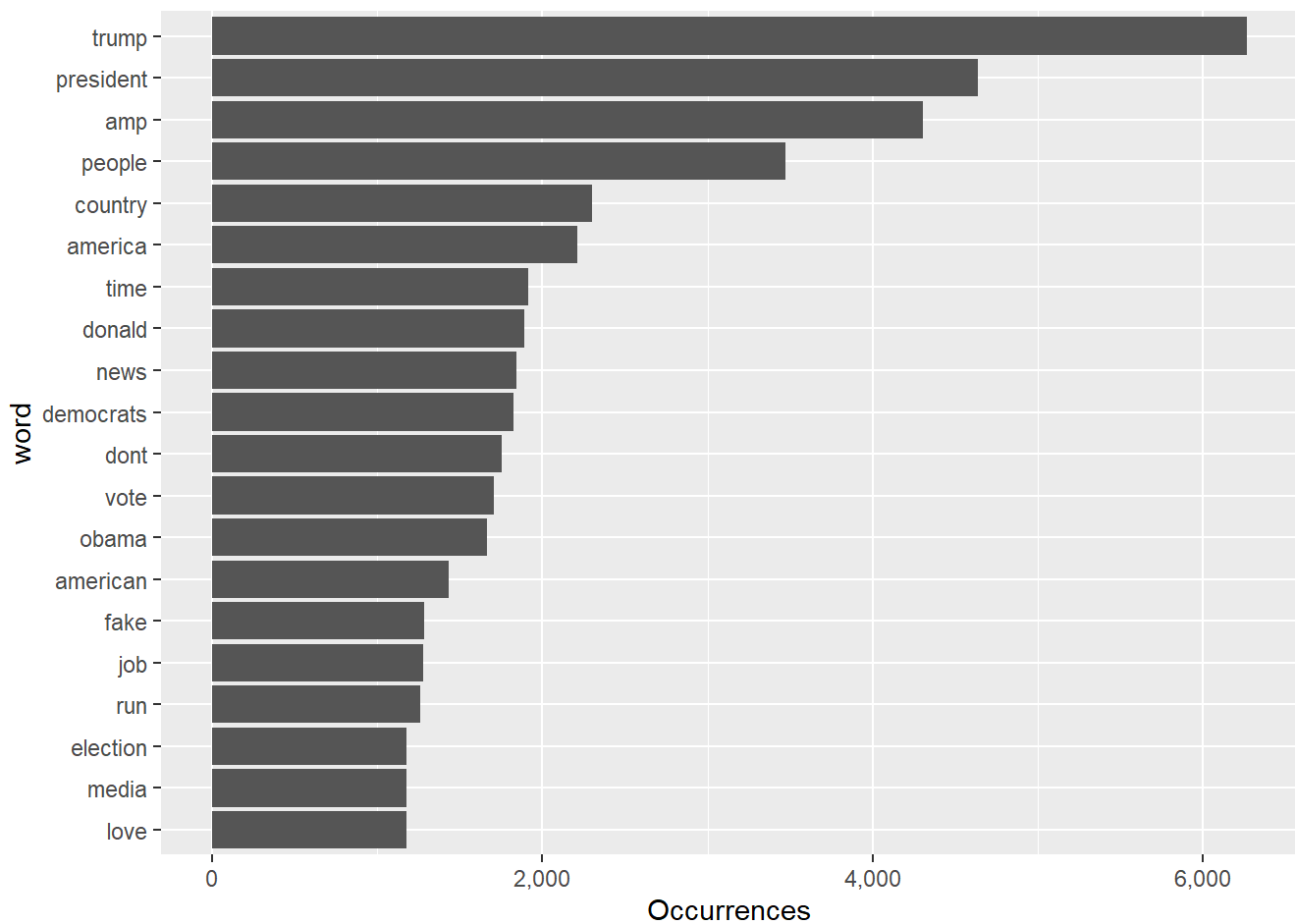
So what are the most commonly tweeted word stems?

```
tweet_words %>%
  count(word) %>%
  arrange(-n)
```

```
## # A tibble: 45,221 × 2
##   word          n
##   <chr>      <int>
## 1 trump      6269
## 2 president  4637
## 3 amp        4306
## 4 people     3475
## 5 country    2302
## 6 america    2211
## 7 time       1913
## 8 donald     1891
## 9 news       1842
## 10 democrats 1824
## # i 45,211 more rows
```

Let's plot the 20 most frequently used words in descending order using a barplot.

```
tweet_words %>%
  count(word, sort = TRUE) %>%
  head(20) %>%
  mutate(word = reorder(word, n)) %>%
  ggplot(aes(word, n)) +
  geom_bar(stat = "identity") +
  ylab("Occurrences") +
  scale_y_continuous(label=comma) +
  coord_flip()
```



Interesting. But we want to know how twitter use changed, if at all, over time and in response to being elected president. So let's start by defining a variation that is `PostPresident` defined to be tweets after being projected as the winner of the 2016 Presidential election.

NOTE: You could also look at other variation (e.g., years, pre/post certain events, etc.). There are lots of opportunities to expand/refine this! Try some!

```
tweet_words <- tweet_words %>%
  mutate(PostPresident = Tweeting.date > as.Date("2016-11-03"))
```

To compare lets compare the top 10 word stems that were tweeted pre-presidency.

```
tweet_words %>%
  filter(PostPresident == FALSE) %>%
  select(word) %>%
  count(word) %>%
  top_n(10, wt= n) %>%
  arrange(-n)
```

```
## # A tibble: 10 × 2
##   word      n
##   <chr>    <int>
## 1 trump    4192
## 2 amp      2218
## 3 president 1757
## 4 donald   1648
## 5 people   1265
## 6 obama    1178
## 7 america  1110
## 8 run      1036
## 9 dont     978
## 10 time    949
```

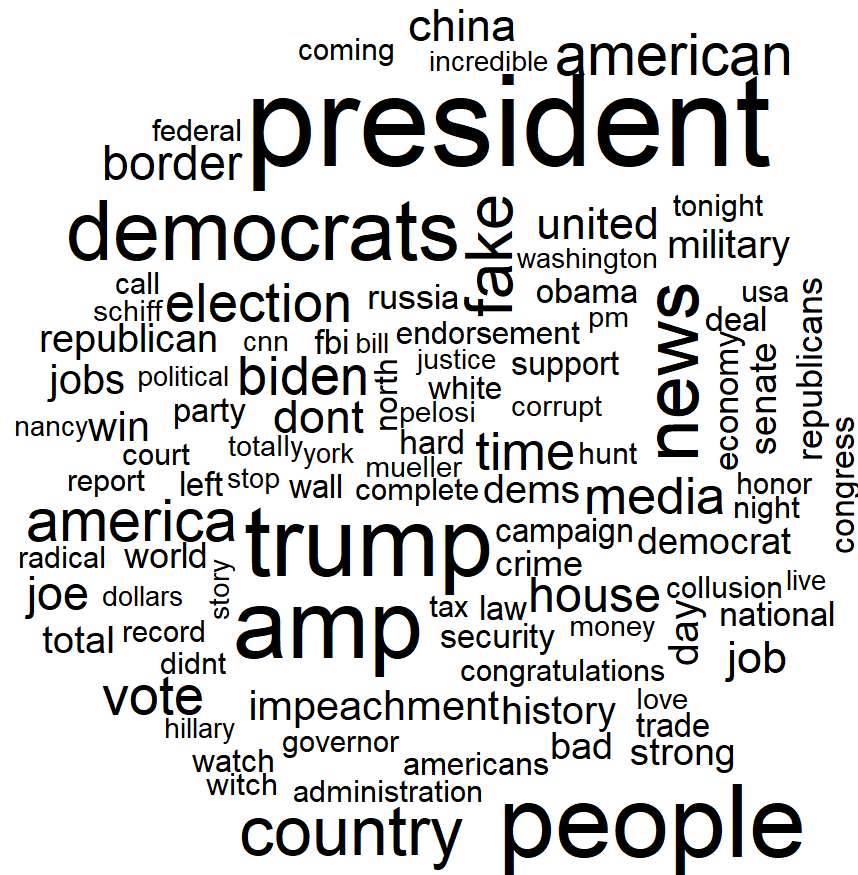
And the top 10 stems tweeted post-presidency.

```
tweet_words %>%
  filter(PostPresident == TRUE) %>%
  select(word) %>%
  count(word) %>%
  top_n(10, wt= n) %>%
  arrange(-n)
```

```
## # A tibble: 10 × 2
##   word      n
##   <chr>    <int>
## 1 president 2880
## 2 people    2210
## 3 amp       2088
## 4 trump     2077
## 5 democrats 1736
## 6 news      1570
## 7 country   1357
## 8 fake      1265
## 9 america   1101
## 10 american 1089
```

Putting together in a nicer table using `group_by`.

```
tweet_words %>%
  group_by(PostPresident) %>%
  select(word) %>%
  count(word) %>%
  top_n(10, wt= n) %>%
  arrange(-n) %>%
  summarise(top_words = str_c(word, collapse = ", "))
```

But what about variation over time? Lots of events happened every year and looking at all tweets before 2016 compared to all of the tweets after Election Day 2016 may lose important nuance and variation. So let's look at the 10 most frequently tweeted words each year.

```
tweet_words %>%
  group_by(Tweeting.year) %>%
  select(word) %>%
  count(word) %>%
  top_n(10, wt= n) %>%
  arrange(-n) %>%
  summarise(top_words = str_c(word, collapse = ", ")) %>%
  knitr::kable()
```

Tweeting.year	top_words
2009	donald, trump, httptinyurlcompqpfvm, champion, trumps, book, watch, david, dont, enter, happy, read, signed
2010	pm, apprentice, trump, nbc, miss, tonight, fantastic, night, tune, episode, hotel
2011	cont, interview, china, pm, america, trump, book, watch, debt, jobs
2012	cont, obama, amp, trump, china, interview, people, dont, time, discussing
2013	trump, amp, people, donald, obama, president, true, dont, time, love
2014	trump, president, amp, donald, run, obama, country, love, true, dont, vote

Tweeting.year top_words

2015	trump, donald, president, amp, america, run, people, country, love, time
2016	hillary, trump, amp, america, people, clinton, crooked, cruz, vote, join
2017	amp, people, news, fake, america, president, tax, trump, country, american
2018	amp, people, president, trump, country, democrats, news, border, fake, time
2019	president, amp, democrats, trump, people, country, news, impeachment, border, fake
2020	president, trump, people, biden, joe, news, democrats, election, vote, american
2021	georgia, election, votes, people, th, dc, january, senators, vote, republican

An now, how about by hour? What is on President Trump's mind, on average, at every hour of the day?

```
# INSERT CODE HERE
```

Pushing ahead, we could also do hour by pre/post presidency (or year) to see how the content changed. Or perhaps how activity varies across parts of the day by creating periods of time based on the hours (e.g., "late-night", "early morning", "normal work-day"). Here is where you as a data-scientist can propose (and defend!) categorizations that are useful for understanding the variation in the data.

Comparing Word Use Pre/Post Using Log Odds Ratio

So far we have focused on frequency of word use, but another way to make this comparison is to look at the relative "odds" that each word is used pre/post presidency. After all, "Trump" is used by Trump both before and after his presidency so perhaps that is not a great indicator of content. We could instead consider the relative rate at which a word is used Post-Presidency relative to Pre-presidency.

We are going to count each word stem use pre and post-presidency, then select only those words that were used at least 5 times, then `spread` the data so that if a word appears Pre-Presidency but not Post-Presidency (or visa-versa) we will create a matching word with the filled in value of 0, then we are going to ungroup the data so that the observation is now a word rather than a word-timing combination (look to see how the tibble changes before and after the `ungroup()` by running these code snippets separately to see). Then we are going to `mutate_each` to compute the fraction of times a word is used relative to all words (the `.` indicates the particular value of each variable – note that we are adding a `+ 1` to each of those values to avoid errors when taking the log later). We then compute the `ratio` by computing the relative frequency of each word used pre and post presidency and take the log of that ratio because of extreme outliers before arranging the tibble in decreasing value of ratio

So let's compute the log odds ratio for each word pre and post presidency.

```

prepost_ratios <- tweet_words %>%
  count(word, PostPresident) %>%
  filter(sum(n) >= 5) %>%
  spread(PostPresident, n, fill = 0) %>%
  ungroup() %>%
  mutate_each(funs((. + 1) / sum(. + 1)), -word) %>%
  mutate(ratio = `TRUE` / `FALSE`) %>%
  mutate(logratio = log(ratio)) %>%
  arrange(-logratio)

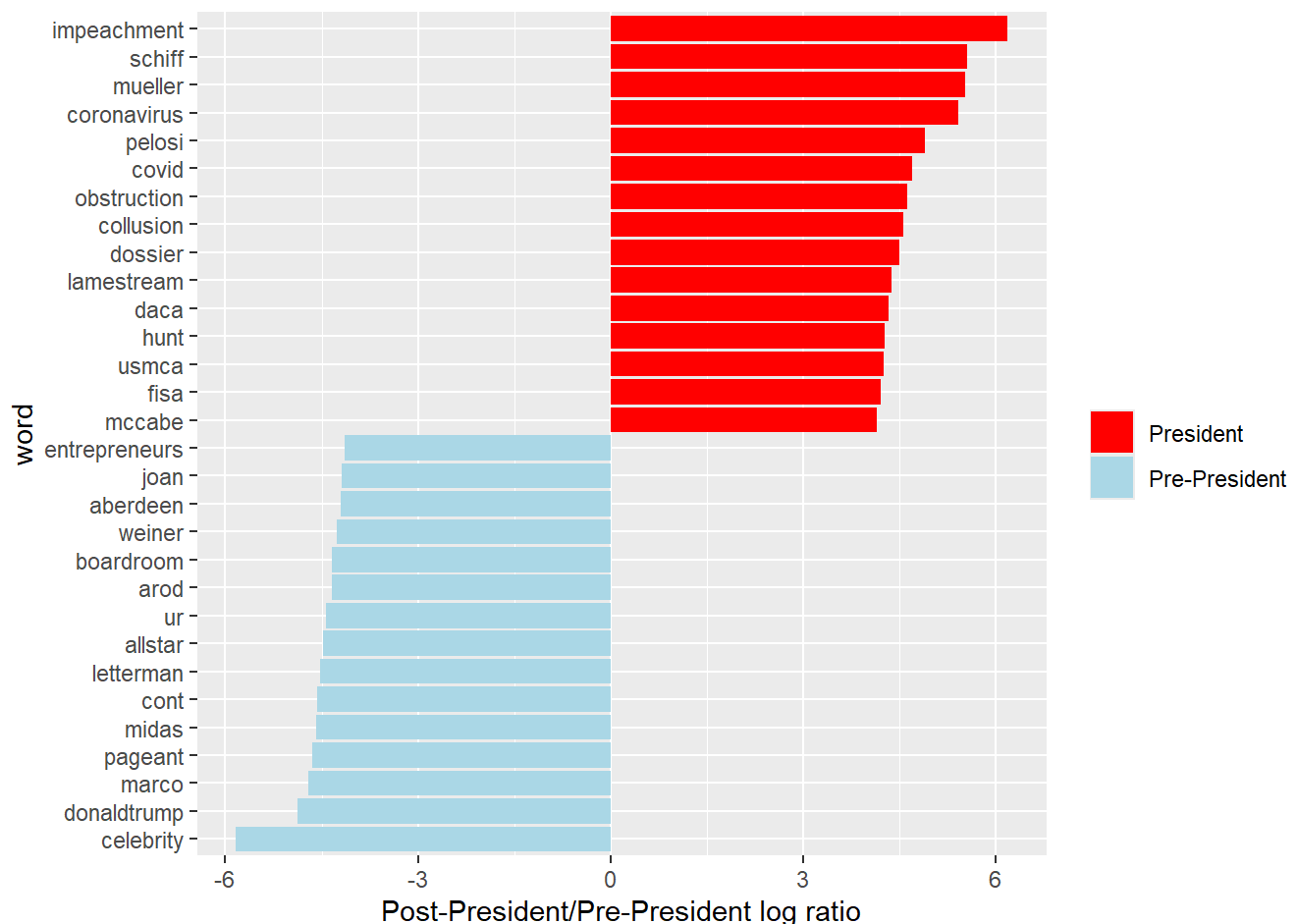
```

Now let's plot the top 15 most distinctive words (according to the log-ratio we just computed) that were tweeted before and after Trump was elected president.

```

prepost_ratios %>%
  group_by(logratio > 0) %>%
  top_n(15, abs(logratio)) %>%
  ungroup() %>%
  mutate(word = reorder(word, logratio)) %>%
  ggplot(aes(word, logratio, fill = logratio < 0)) +
  geom_bar(stat = "identity") +
  coord_flip() +
  ylab("Post-President/Pre-President log ratio") +
  scale_fill_manual(name = "", labels = c("President", "Pre-President"),
                    values = c("red", "lightblue"))

```



You could look at other splits. Pre-post his first impeachment? 2016 versus 2017? Note that the log-ratio is a comparison of a binary condition.

k-means

How else can we summarize/describe data? Cluster Analysis via `kmeans` ?

But using what data? Should we focus on the number of words being used? The proportion of times a word is used in a particular document? Or some other transformation that tries to account for how frequently a word is used in a particular document relative to how frequently it is used in the overall corpus?

We are going to use the text analysis function `bind_tf_idf` that will take a document-term matrix and compute the fraction of times each word is used in each document (`tf` = “term frequency”). It also computes a transformation called tf-idf that balances how frequently a word is used relative to its uniqueness in a document.

For word w in document d we can compute the tf-idf using:

$$tf-idf(w, d) = tf(w, d) \times \log \left(\frac{N}{df(w)} \right)$$

where `tf` is the term frequency (word count/total words), `df(w)` is the number of documents in the corpus that contain the word, and `N` is the number of documents in the corpus. The inverse-document-frequency `idf` for each word w is therefore the number of documents in the corpus `N` over the number of documents containing the word.

NOTE: what is a document? In theory, a document is a tweet. However, tweets are so short that most words only appear once. Furthermore, there are a LOT of tweets written by Trump over his time on the platform. Instead, we will treat a DAY (`Tweeting.date`) as our “document”. Be aware what this implies! We are assuming that Trump’s tweets written on a given day are about the same thing. This is obviously not always true, but it is a reasonable assumption to start with.

So let us create an new document-term-matrix object that also includes the `tf`, `idf` and `tf_idf` associated with each word.

```
# Create the dtm with Tweeting.date as the "document".
dtm <- tweet_words %>%
  count(Tweeting.date, word) %>%
  group_by(word) %>%
  mutate(tot_n = sum(n)) %>%
  ungroup() %>%
  filter(tot_n > 20) # Drop words that appear less than 20 total time across the entire
data

require(tidytext)
dtm.tfidf <- bind_tf_idf(tbl = dtm, term = word, document = Tweeting.date, n = n) # Calc
ulate TF-IDF
dtm.tfidf %>%
  select(word, tf_idf) %>%
  distinct() %>%
  arrange(-tf_idf) %>%
  slice(1:10)
```

```
## # A tibble: 10 × 2
##   word      tf_idf
##   <chr>    <dbl>
## 1 gma      5.21
## 2 generation 4.90
## 3 dead     3.19
## 4 apprentice 2.58
## 5 weekly   2.40
## 6 appearance 2.26
## 7 answers   2.11
## 8 sounds    2.10
## 9 football  2.01
## 10 defeat   1.90
```

So `kmeans` took a matrix where each column was a different variable that we were interested in using to characterize patterns but the data we have is arranged in a one-term-per-document-per-row. To transform the data into the format we require we need to “recast” the data so that each word is a separate variable – meaning that the number of variables is the number of unique word stems.

```
castdtm <- cast_dtm(data = dtm.tfidf, document = Tweeting.date, term = word, value = tf_idf)
```

And now we can run *k*-means on this object! How many `centers` (clusters or *K*) should we choose? This all depends on how many different things we think Trump talks about. For now, we will just use 50. However, recall that we should create an “elbow” plot like we did in the previous lecture, and choose *k* based on where the reductions in errors are smaller.

```
set.seed(42)
km_out <- kmeans(castdtm,
                 centers = 50, # 50 "topics"
                 nstart = 5) # Set nstart = 5 to make sure this doesn't take forever!
```

So how many documents are associated with each cluster?

```
table(km_out$cluster)
```

```
##
##   1    2    3    4    5    6    7    8    9   10   11   12   13   14   15   16
## 12    1    1    4   16   36  156    1    1    9    2   34 3066    2    1    1
## 17   18   19   20   21   22   23   24   25   26   27   28   29   30   31   32
##   1    5    1    4    1    1   15    1    1    1    1    4    8    4    3    2
## 33   34   35   36   37   38   39   40   41   42   43   44   45   46   47   48
##   1    3    1   12    5    3    1    7    4    1    3   17    1    2   10    1
##  49   50
##  17    7
```

So let's tidy it up to see the centroids – here mean frequency– associated with each word in each cluster.

```
require(broom)
tidy(km_out)
```

```
## # A tibble: 50 × 3,180
##       david donald late letterman list night ten tonight top
##       <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
##  1 0      0.00970 0      0 0      0.0783 0      0.0898 0
##  2 0      0      0      0 0      0      0      0      0
##  3 0      0      0      0 0      0      0      0      0
##  4 0      0      0      0 0      0      0      0      0
##  5 0      0      0      0 0      0.00861 0      0.0297 0
##  6 0      0.250 0      0 0.0209 0.0170 0.0201 0      0.0112
##  7 0.000647 0.00121 0.00114 0 0.000271 0.00859 0      0.00427 0.00160
##  8 0      0      0      0 0      0      0      0      0
##  9 0      0      0      0 0      0      0      0      0.607
## 10 0      0.0559 0      0 0      0      0      0      0
## # i 40 more rows
## # i 3,171 more variables: trump <dbl>, tune <dbl>, watch <dbl>,
## #   apprentice <dbl>, book <dbl>, celebrity <dbl>, champion <dbl>,
## #   discuss <dbl>, morning <dbl>, tomorrow <dbl>, view <dbl>, finale <dbl>,
## #   financial <dbl>, funny <dbl>, learned <dbl>, post <dbl>, build <dbl>,
## #   fired <dbl>, id <dbl>, ive <dbl>, miss <dbl>, usa <dbl>, walls <dbl>,
## #   discussing <dbl>, interview <dbl>, listen <dbl>, sense <dbl>, ...
```

Very hard to summarize given that there are 3180 columns! (Good luck trying to graph that!)

How can we summarize? Want to `gather()` to convert the data to **long** form.

```
km_out_tidy <- tidy(km_out) %>%
  gather(word, mean_tfidf, -size, -cluster, -withinss) %>% # Convert to long data (don't add
  "size", "cluster", and "withinss" to the new "word" column! These are part of the tidy()
  output!)
  mutate(mean_tfidf = as.numeric(mean_tfidf)) # Calculate average TF-IDF
km_out_tidy
```

```
## # A tibble: 158,850 × 5
##       size withinss cluster word mean_tfidf
##       <int> <dbl> <fct> <chr> <dbl>
##  1     12    12.6 1      david 0
##  2      1      0 2      david 0
##  3      1      0 3      david 0
##  4      4    7.12 4      david 0
##  5     16   12.4 5      david 0
##  6     36   43.0 6      david 0
##  7    156   54.5 7      david 0.000647
##  8      1      0 8      david 0
##  9      1      0 9      david 0
## 10      9    9.95 10     david 0
## # i 158,840 more rows
```

This tells us that the word “apprentice” is only weakly associated with topic (“cluster”) 3 (0.000296), but is more strongly associated with topic 8 (0.0193) and topic 10 (0.0365). We can also see how many documents (i.e., days) are associated with each topic. Topic 1 only appears once, while topic 3 appears 1,076 times.

Let’s try plotting just the third topic to better understand what it is about. To do this, we want to look at the top 10 highest scoring words according to `mean_tfidf`.

```
km_out_tidy %>%
  filter(cluster %in% 3) %>%
  group_by(cluster) %>%
  arrange(-mean_tfidf) %>%
  slice(1:10)
```

```
## # A tibble: 10 × 5
## # Groups:   cluster [1]
##   size withinss cluster word      mean_tfidf
##   <int>   <dbl> <fct>   <chr>         <dbl>
## 1     1     0 3 apprentice 2.58
## 2     1     0 3 david      0
## 3     1     0 3 donald     0
## 4     1     0 3 late       0
## 5     1     0 3 letterman 0
## 6     1     0 3 list       0
## 7     1     0 3 night      0
## 8     1     0 3 ten        0
## 9     1     0 3 tonight    0
## 10    1     0 3 top        0
```

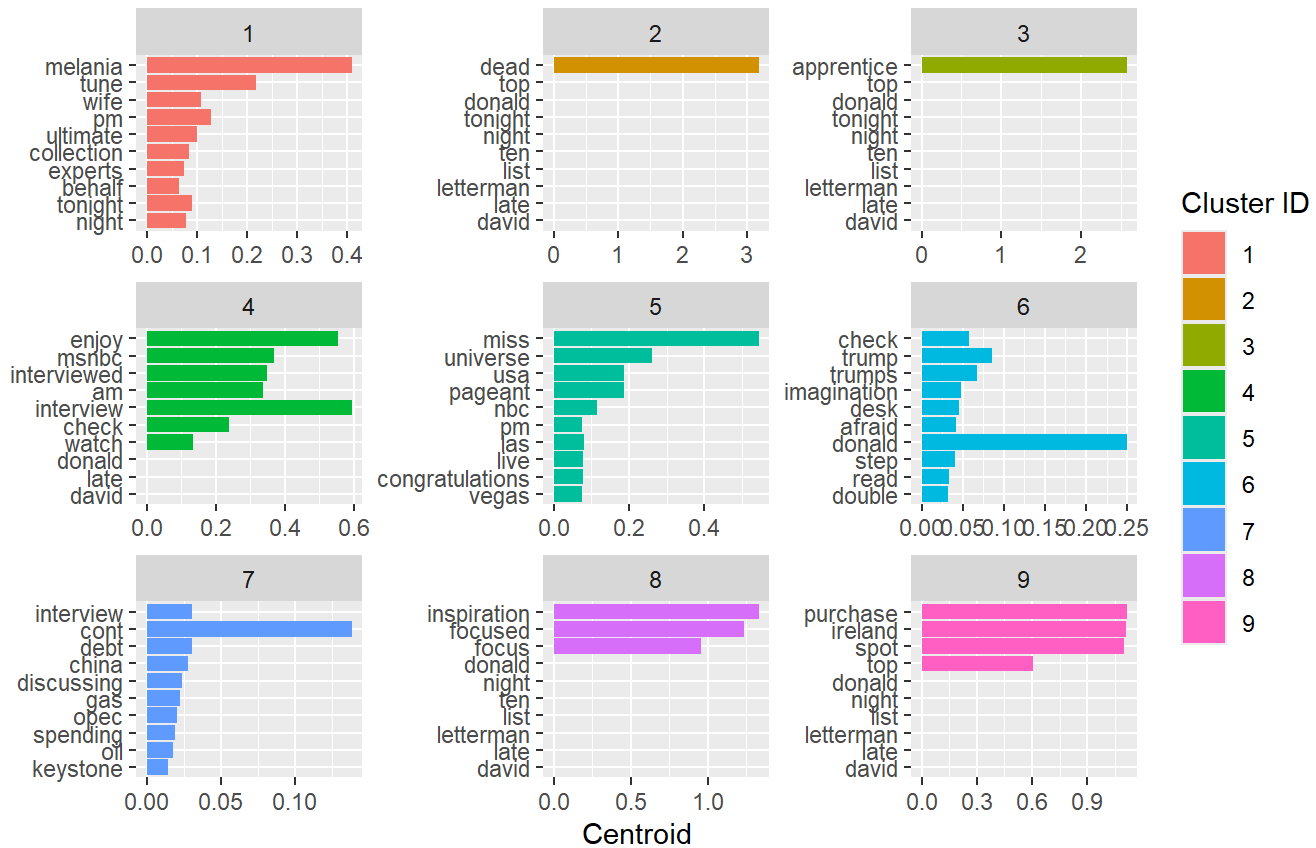
This appears to be about domestic politics! For those who are familiar with Trump’s time in office, he would frequently talk about fake news and the media, as well as his flagship policy: the border wall with Mexico.

We can turn this into a plot for easier visualization (and look at multiple topics at once).

```
km_out_tidy %>%
  filter(cluster %in% 1:9) %>%
  group_by(cluster) %>%
  arrange(-mean_tfidf) %>%
  slice(1:10) %>%
  ggplot(aes(x = mean_tfidf, y = reorder(word, mean_tfidf),
            fill = factor(cluster))) +
  geom_bar(stat = 'identity') +
  facet_wrap(~cluster, scales = 'free') +
  labs(title = 'k-means Clusters',
       subtitle = 'Clustered by TF-IDF',
       x = 'Centroid',
       y = NULL,
       fill = 'Cluster ID')
```

k-means Clusters

Clustered by TF-IDF



We can also assign each topic to the “documents” it is associated with. To do this, we need to create a new dataset as follows:

- Get the document names from the `castdtm` object. These are stored in the `dimnames` variable under `Docs`.
- Get the cluster (topics) for each document from the `km_out` object. These are stored in the `cluster` variable.
- R automatically converted the document names to character representations of the numeric version of the data. We need to convert these back to dates using `as.Date()` combined with `as.numeric()`. Since these are stored as raw numbers, we must specify the `origin = "1970-01-01"` in order for the `as.Date()` function to work properly.

```
doc_cluster <- data.frame(Tweeting.date = castdtm$dimnames$Docs, # Get the document names
                          cluster = km_out$cluster) %>% # Get the topics from the km_out object
  as_tibble() %>%
  mutate(Tweeting.date = as.Date(as.numeric(Tweeting.date), origin = '1970-01-01')) # Convert the document names back to date formats

doc_cluster
```

```
## # A tibble: 3,492 × 2
##   Tweeting.date cluster
##   <date>         <int>
## 1 2009-05-04         18
## 2 2009-05-05         12
## 3 2009-05-08         18
## 4 2009-05-12          6
## 5 2009-05-13         10
## 6 2009-05-14          6
## 7 2009-05-15         28
## 8 2009-05-16          6
## 9 2009-05-17          6
## 10 2009-05-18         6
## # i 3,482 more rows
```

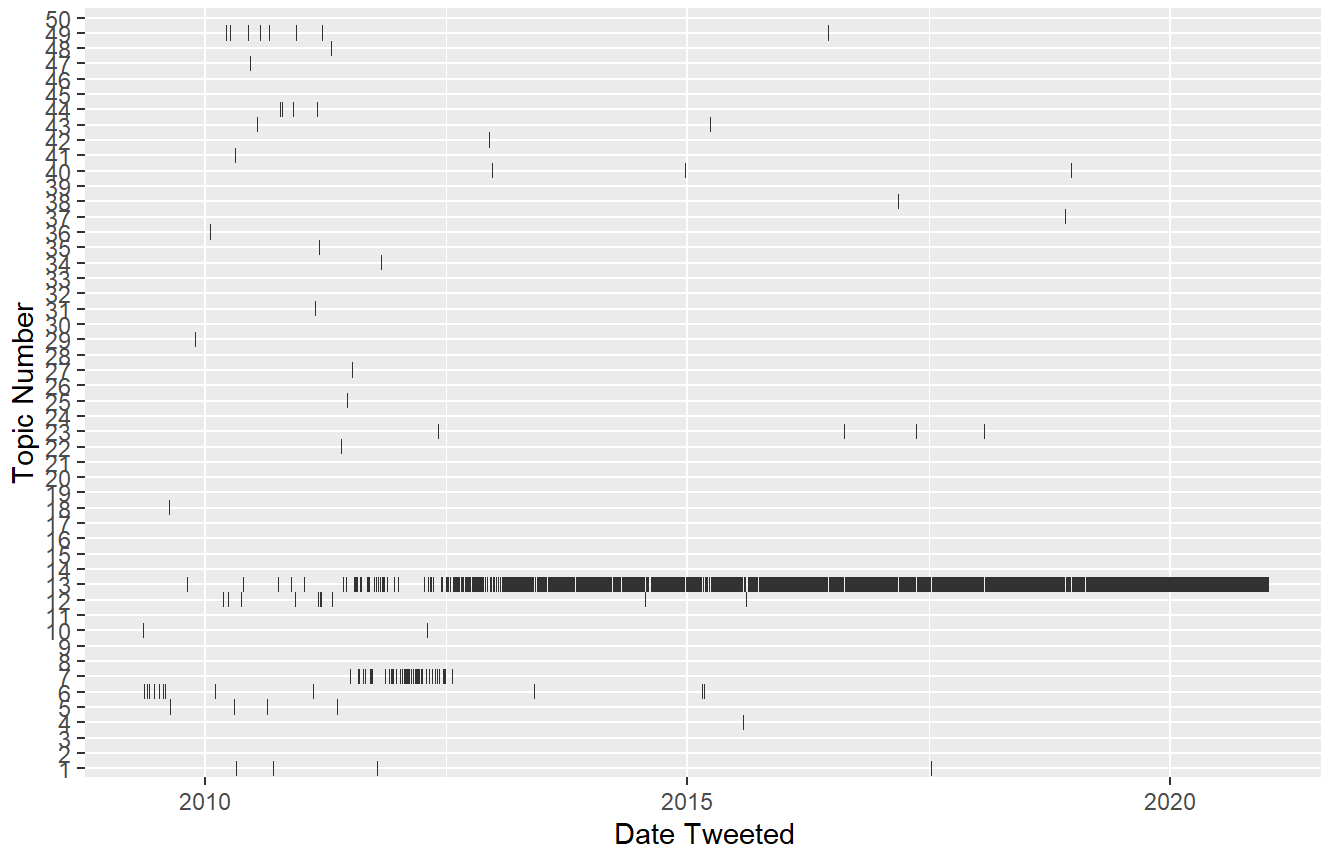
So topic 35 was the focus of Trump's tweets on his first two days on the platform, following by 6 days where he emphasized topic 21.

Let's plot these to make it easier to see patterns.

```
doc_cluster %>%
  ggplot(aes(x = Tweeting.date, y = factor(cluster))) +
  geom_tile() +
  labs(title = 'Topics by Date',
       subtitle = 'Donald Trump Twitter Account',
       x = 'Date Tweeted',
       y = 'Topic Number')
```


Topics by Date

Donald Trump Twitter Account

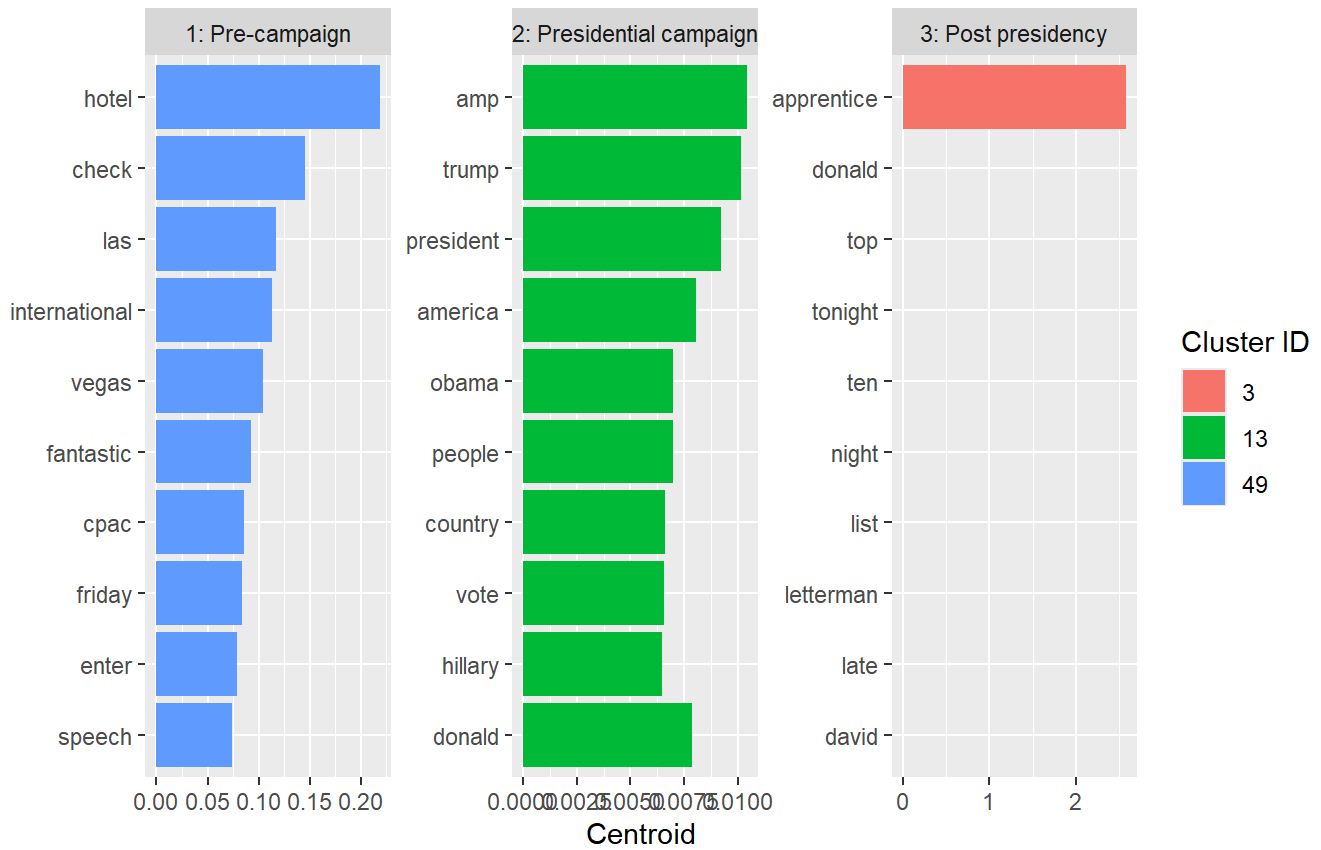


There are basically 3 (maybe 4?) core topics from our data. Topic 49 is Trump's main focus prior to 2016 when he starts campaigning. Then it shifts to topic 13 throughout the campaign, before settling on topic 3 during his presidency. Let's look at these 3 topics!

```
km_out_tidy %>%
  filter(cluster %in% c(3,13,49)) %>%
  group_by(cluster) %>%
  arrange(-mean_tfidf) %>%
  slice(1:10) %>%
  mutate(topic = ifelse(cluster == 3, '3: Post presidency',
                        ifelse(cluster == 13, '2: Presidential campaign', '1: Pre-campaign')) %>%
  ggplot(aes(x = mean_tfidf, y = reorder(word, mean_tfidf),
            fill = factor(cluster))) +
  geom_bar(stat = 'identity') +
  facet_wrap(~topic, scales = 'free') +
  labs(title = 'k-means Clusters',
       subtitle = 'Clustered by TF-IDF',
       x = 'Centroid',
       y = NULL,
       fill = 'Cluster ID')
```

k-means Clusters

Clustered by TF-IDF



Amazing! Using just k -means with a bag-of-words of Trump's tweets, we have a clear idea of what he talked about during different periods!